

Memory Architecture & Addressing Principles

The load/store model, address computation, data sizing,
and memory hierarchy interactions in AArch64

Dr. J. Burns

Computer Systems Architecture — AArch64 Reference Platform

Topic 5 Outline

1. Memory Access and Computation

- Why AArch64 separates memory transfers from arithmetic operations.
- How values move between memory and the register file.

2. Data Types, Widths, and Conversions

- Handling bytes, halfwords, words, and doublewords cleanly.
- Choosing zero-extension or sign-extension when loading sub-word data.

3. Address Calculation

- Immediate offsets, pre/post-indexing, and scaled array indexing.
- How each addressing form changes, or does not change, the base register.

4. Applied Access Patterns

- Practical array traversal and indexed-array access patterns.
- Choosing the addressing form that matches the data-access pattern.

PART 1

The Load/Store Architectural Boundary

The Fundamental Memory Challenge

- Registers are fast but limited in number; memory holds much more data but is accessed through explicit transfer instructions.
- **Architectural Question:** How should the instruction set organize data movement between memory and the register file?
- AArch64 uses a strict **load/store discipline**: ALU instructions operate on registers, while load/store instructions access memory.

The Architectural Rule

ALU instructions process *only* register operands. Memory access is performed by load/store-family instructions.

Separating Computation from Data Movement

Architectural Separation

- Arithmetic instructions do not directly access data memory.
- Load instructions copy values from memory into registers.
- Store instructions copy register values back to memory.

Typical Data Flow

- 1 **Load:** bring data into registers.
- 2 **Compute:** operate on register values.
- 3 **Store:** write the result back if needed.



Addressing Memory: Bytes, Words, and Doublewords

- Memory is a linear sequence of **8-bit bytes**; each byte has its own address.
- The instruction/access size determines the transfer width:

Data Type	Width	Load Syntax	Store Syntax
Byte	8-bit	<code>ldrb wt, [xn]</code>	<code>strb wt, [xn]</code>
Halfword	16-bit	<code>ldrh wt, [xn]</code>	<code>strh wt, [xn]</code>
Word	32-bit	<code>ldr wt, [xn]</code>	<code>str wt, [xn]</code>
Doubleword	64-bit	<code>ldr xt, [xn]</code>	<code>str xt, [xn]</code>

The base register (xn) holds the memory pointer enclosed in brackets: [xn].

PART 2

Data Sizing & Extension Rules

Sub-Word Loads and Extension

- Registers are 64 bits wide, but memory values may be only 8, 16, or 32 bits.
- When a smaller value is loaded, the processor must fill the remaining upper bits.
- The correct choice depends on whether the memory value should be treated as **unsigned** or **signed**.

Zero-Extension

Use for unsigned data. Upper bits are filled with 0s.

Sign-Extension

Use for signed data. The sign bit is copied into the upper bits.

Sign vs. Zero Extension: Side-by-Side

Memory Byte at Address x1 = 0x80 (Binary: 1000 0000)

The same 8-bit pattern means either unsigned 128 or signed -128, depending on the load instruction.

Instruction	Value in x0	Meaning
ldrb w0, [x1]	0x00000000_00000080	Zero-extends the byte; interpreted as unsigned +128.
ldrsb x0, [x1]	0xffffffff_ffffff80	Sign-extends the byte; preserves signed value -128.

Using a zero-extending load for signed negative data can turn a negative value into a large positive one.

Storing Data: Truncation Rules

- Stores move data from a register into memory.
- The instruction width determines how many bytes from the register are written:

Instruction	Bits Read from Register	Bytes Written to Memory
<code>strb wt, [xn]</code>	Bits [7:0]	Writes exactly 1 byte
<code>strh wt, [xn]</code>	Bits [15:0]	Writes exactly 2 bytes
<code>str wt, [xn]</code>	Bits [31:0]	Writes exactly 4 bytes
<code>str xt, [xn]</code>	Bits [63:0]	Writes full 8 bytes (doubleword)

Storing a byte or halfword modifies *only* the targeted memory locations; neighboring bytes are left unchanged.

PART 3

AArch64 Addressing Modes

Addressing Modes

- An **addressing mode** is how a load/store instruction forms the memory address it will access.
- The resulting address is the **effective address**: the actual byte address used by the load or store.

Access Pattern	Addressing Form	Typical Use
Fixed offset from a base pointer	<code>[xn, #imm]</code>	Struct fields, known offsets
Step through consecutive elements	<code>[xn], #imm</code>	Array traversal
Base plus scaled index	<code>[xn, xm, lsl #k]</code>	<code>arr[i]</code> access

The key question is whether the offset is fixed, index-based, or part of a pointer update.

Mode 1: Base Register with Immediate Offset

- Adds a fixed constant to a base pointer without modifying the base register.
- **Syntax:** `ldr xt, [xn, #offset]` $\implies EA = x_n + \text{offset}$.

Base Address (`x0`)

+Offset



Target Memory Address

Example: Reading Struct Fields

```
ldr x1, [x0, #0]    // Read field 1 at offset 0
ldr x2, [x0, #8]     // Read field 2 at offset 8
ldr w3, [x0, #16]    // Read 32-bit field at offset 16
```

Scaled Immediate Offsets

- Some load/store offsets are scaled by the size of the access.
- For a 64-bit load, an offset of #8 means one doubleword past the base address.
- For a 32-bit load, an offset of #4 means one word past the base address.

Conceptual Rule

The offset tells the instruction where the desired object is relative to the base pointer. The instruction width tells the processor how many bytes to transfer.

Mode 2: Pre-Indexed Addressing (Update Before Access)

- Updates the base register **before** reading or writing memory.
- **Syntax:** `ldr xt, [xn, #offset]!` (Exclamation mark indicates update).

Step-by-Step Action

- 1 $x_n \leftarrow x_n + \text{offset}$ (Update pointer)
- 2 Access memory at the **new** address in x_n .

Primary Use Case: Access After Pointer Adjustment

```
ldr x0, [x1, #-8]!
```

Moves the base pointer back by 8 bytes, then loads from the updated address.

Mode 3: Post-Indexed Addressing (Update After Access)

- Reads or writes memory at the current address, then updates the base pointer **afterwards**.
- **Syntax:** `ldr xt, [xn], #offset` (Offset written outside the brackets).

Step-by-Step Action

- 1 Access memory at the **current** address in x_n .
- 2 $x_n \leftarrow x_n + \text{offset}$ (Advance pointer)

Primary Use Case: Array Stepping

```
ldr x0, [x1], #8
```

Loads data from the current address in x1, then automatically advances x1 to the next 8-byte element.

Comparing the Three Offset Modes

Mode	Syntax Pattern	Address Accessed	Base Register After
Standard Offset	$[x_n, \#8]$	$x_n + 8$	x_n (Unchanged)
Pre-Indexed	$[x_n, \#8]!$	$x_n + 8$	$x_n + 8$ (Updated)
Post-Indexed	$[x_n], \#8$	x_n	$x_n + 8$ (Updated)

Assume $x_1 = 0x1000$ Initially

<code>ldr x0, [x1, #8]</code>	$\implies$ Reads from <code>0x1008</code> ;	x_1 remains <code>0x1000</code>
<code>ldr x0, [x1, #8]!</code>	$\implies$ Reads from <code>0x1008</code> ;	x_1 becomes <code>0x1008</code>
<code>ldr x0, [x1], #8</code>	$\implies$ Reads from <code>0x1000</code> ;	x_1 becomes <code>0x1008</code>

Mode 4: Register Offset with Scaling

- Uses an index register (x_m) multiplied by element size to access array items:

```
ldr xt, [xn, xm, lsl #shift]
```

- The instruction encodes the scaled register offset directly, though implementation latency depends on the processor.

Array Element Type	Assembly Instruction	Effective Address
Byte (uint8)	ldrb w0, [x1, x2]	$EA = x_1 + x_2$
Halfword (uint16)	ldrh w0, [x1, x2, lsl #1]	$EA = x_1 + (x_2 \times 2)$
Word (uint32)	ldr w0, [x1, x2, lsl #2]	$EA = x_1 + (x_2 \times 4)$
Doubleword (uint64)	ldr x0, [x1, x2, lsl #3]	$EA = x_1 + (x_2 \times 8)$

Directly implements high-level array access: `val = arr[i];`.

PART 4

Multi-Register Transfers & Applied Access Patterns

Paired Transfers: `ldp` and `stp`

- `ldp` and `stp` transfer **two adjacent registers** with one architectural instruction.
- They are useful when nearby values are loaded or stored together.

Instruction	Action	Typical Use
<code>ldp x0, x1, [x2]</code>	Load two 64-bit values	Adjacent data values
<code>stp x0, x1, [x2]</code>	Store two 64-bit values	Adjacent output values
<code>ldp x2, x3, [x1, #16]</code>	Load pair at an offset	Copying or scanning data

Paired transfers reduce instruction count when the program naturally works with adjacent values.

Worked Pattern 1: Stepping Through an Array

Goal: Sum N 64-bit Integers

AArch64 Implementation (Using Post-Indexed Addressing)

```
// x0 = array pointer, x1 = count n
sum_loop:
    mov x2, xzr          // sum = 0
    cbz x1, .Ldone       // Exit if count == 0

.Lbody:
    ldr x3, [x0], #8      // Load element AND increment pointer by 8
    add x2, x2, x3        // sum += element
    sub x1, x1, #1        // count-- (flags not needed)
    cbnz x1, .Lbody       // Loop if count != 0

.Ldone:
    mov x0, x2            // Return sum in x0
    ret
```

Post-indexing avoids a separate pointer-update instruction such as `add x0, x0, #8` inside the loop.

Worked Pattern 2: Indexed Array Search

Goal: Check Element at Index i in Array

AArch64 Implementation (Using Scaled Register Offset)

// x0 = array pointer, x1 = index i, x2 = target value

find_element:

```
    ldr x3, [x0, x1, lsl #3]    // Load arr[i] using Base + (i * 8)
    cmp x3, x2                  // Compare arr[i] with target
    cset w0, eq                 // w0 = 1 if match, else 0
    ret
```

Combines index scaling and load into a single instruction without modifying the base pointer x0.

Addressing Mode Selection Guide

Programming Scenario	Addressing Mode	Example Instruction
Accessing a field at a fixed offset	Base + immediate offset	<code>ldr x1, [x0, #8]</code>
Loading the next array element	Post-indexed update	<code>ldr x2, [x0], #8</code>
Stepping backward through data	Pre-indexed update	<code>ldr x2, [x0, #-8]!</code>
Accessing <code>arr[i]</code>	Scaled register offset	<code>ldr x2, [x0, x1, lsl #3]</code>
Moving two adjacent values	Paired transfer	<code>ldp x2, x3, [x0]</code>

Common Memory Access Mistakes

- **Wrong Sizing:** Using `ldr xt` instead of `ldr wt` on an array of 32-bit integers reads too many bytes.
- **Missing Sign Extension:** Using `ldrb` instead of `ldrsh` on signed byte data changes negative values into positive ones.
- **Incorrect Shift Scale:** Using `lsl #2` instead of `lsl #3` on an array of 64-bit values computes the wrong address.
- **Accidental Base Update:** Using pre-indexed or post-indexed addressing when the original pointer still needs to be preserved.

SUMMARY & ASSESSMENT

Review and Self-Test Questions

Topic 5 Summary

- **Load/Store Discipline:** ALU instructions operate on registers; load/store-family instructions move data between registers and memory.
- **Data Widths & Extensions:** The instruction width controls how many bytes are transferred; signed sub-word data needs sign-extension.
- **Core Addressing Modes:**
 - Standard offset ($[xn, \#imm]$): base register is unchanged.
 - Pre-indexed ($[xn, \#imm]!$): base updated before the access.
 - Post-indexed ($[xn], \#imm$): base updated after the access.
 - Scaled register offset ($[xn, xm, \ll \#3]$): base plus scaled index for arrays.
- **Applied Access Patterns:** Sequential traversal often uses post-indexing; indexed array access often uses a scaled register offset.

Self-Test Questions: Sizing & Addressing Modes

- ❶ **Sign Extension vs. Zero Extension:** Memory byte at address `0x2000` holds value `0xfe` (-2 in signed 8-bit). What value resides in register `x0` after:
 - `ldrb w0, [x1]` (where $x_1 = 0x2000$)
 - `ldrsh x0, [x1]` (where $x_1 = 0x2000$)
- ❷ **Addressing Mode Mechanics:** If register `x1` holds address `0x3000`, what address is read and what is the final value in `x1` for:
 - `ldr x0, [x1, #8]`
 - `ldr x0, [x1, #8]!`
 - `ldr x0, [x1], #8`
- ❸ **Array Element Indexing:** Given base array pointer in `x0` and index i in `x1`, write a single instruction to load a 32-bit integer `arr[i]` into `w2`.

Self-Test Questions: Applied Addressing Patterns

- ❶ **Sequential Traversal:** Register `x0` points to the current element of a 64-bit array. Write one load instruction that reads the current element into `x2` and then advances `x0` to the next element.
- ❷ **Scaled Indexing:** Register `x0` holds the base address of a 32-bit integer array and `x1` holds index i . Write one instruction that loads `arr[i]` into `w2`.
- ❸ **Mode Selection:** Which addressing form best matches each case: a fixed struct field, sequential traversal, and `arr[i]` access?

Looking Ahead

Next Lecture Preview

Our Next Topic: AArch64 Subroutines, Calling Conventions, & the Stack

- **Subroutine Calls:** Using `bl`, `blr`, and `ret`.
- **Calling Convention:** Passing arguments, returning values, and preserving registers.
- **Stack Frames:** Applying stack alignment, saved registers, and frame layout in real subroutines.